

Global Engineering Solutions

FEST®

FEST Library Reference Manual

CSeqObject		
1.	BEGIN_OBJECT_ENUMERATION/ END_OBJECT_ENUMERATION	Page7
2.	RecipeName	Page9
3.	GetName	Page10
4.	ObjType	Page11
CIOObject		
1.	GetName	Page13
2.	GetFormat	Page14
3.	GetIDs	Page15
4.	GetType	Page16
5.	GetMode	Page17
6.	GetElapsedTime	Page18
7.	GetIODriver	Page19
8.	Poll	Page20
9.	PollIfElapsed	Page21
CEnum & CDIO		
1.	GetName	Page23
2.	GetMaxValue	Page24
3.	GetEnumStr	Page25
4.	GetEnumStrAll	Page26
5.	GetEnumValue	Page27
6.	GetEnum	Page28
7.	GetMinMax	Page29
8.	SetMinMax	Page30
9.	Compare	Page31
10.	Less	Page32
11.	LessEqual	Page33
12.	Equal	Page34
13.	NotEqual	Page35
14.	GreaterEqual	Page36
15.	Greater	Page37
16.	InRange	Page38
17.	InRangeEx	Page39
18.	InRangeExEnd	Page40
19.	InRangeExBegin	Page41
20.	Read	Page42



21.	ReadWithIDs	Page43
22.	ReadIfElapsed	Page44
23.	ReadAndGetStr	Page45
24.	ReadAndGetStrIfElapse d	Page46
25.	ReadEnum	Page47
26.	ReadEnumIfElapsed	Page48
27.	ReadFromMem	Page49
28.	ReadAndGetStrFromMe m	Page50
29.	ReadEnumFromMem	Page51
30.	Write	Page52
31.	WriteWithIDs	Page53
32.	WriteTh	Page54
33.	WriteThWithIDs	Page55
34.	WriteEnum	Page56
35.	WriteToMem	Page57
36.	WriteToMemDirect	Page58
37.	Increase	Page59
38.	Decrease	Page60
39.	Multiply	Page61
40.	Divide	Page62
	CLookUp & CAIO	
1.	GetName	Page64
2.	GetDepth	Page65
3.	GetConversionMode	Page66
4.	GetLookUp	Page67
5.	GetUnit	Page68
6.	GetMinMax	Page69
7.	SetMinMax	Page70
8.	Compare	Page71
9.	Less	Page72
10.	LessEqual	Page73
11.	Equal	Page74
12.	NotEqual	Page75
13.	GreaterEqual	Page76
14.	Greater	Page77
15.	InRange	Page78



16.	InRangeEx	Page79
17.	InRangeExEnd	Page80
18.	InRangeExBegin	Page81
19.	Read	Page82
20.	ReadWithIDs	Page83
21.	ReadIfElapsed	Page84
22.	ReadAndGetStr	Page85
23.	ReadAndGetStrIfElapse d	Page86
24.	ReadFromMem	Page87
25.	ReadAndGetStrFromMe m	Page88
26.	Write	Page89
27.	WriteWithIDs	Page90
28.	WriteTh	Page91
29.	WriteThWithIDs	Page92
30.	WriteToMem	Page93
31.	Increase	Page94
32.	Decrease	Page95
33.	Multiply	Page96
34.	Divide	Page97
	CSIO	
1.	Compare	Page99
2.	Equal	Page100
3.	NotEqual	Page101
4.	Read	Page102
5.	ReadWithIDs	Page103
6.	ReadIfElapsed	Page104
7.	ReadFromMem	Page105
8.	Write	Page106
9.	WriteWithIDs	Page107
10.	WriteTh	Page108
11.	WriteThWithIDs	Page109
12.	WriteToMem	Page110
13.	WriteToMemDirect	Page111
14.	WriteCount	Page112
	CSeqObject	
1.	GetName	Page114



2.	Load		Page115
3.	Kill		Page116
4.	Status		Page117
5.	Status2		Page118
6.	Write		Page119
7.	Write2		Page120
8.	WriteF		Page121
9.	Run		Page122
10.	RunF		Page123
11.	RunTh		Page124
12.	RunThF		Page125
13.	RunAny		Page126
14.	RunAnyF		Page127
15.	RunAnyTh		Page128
16.	RunAnyThF		Page129
17.	Abort		Page130
18.	SendEventMsg		Page131
19.	SendEventMsgF		Page132
20.	CallFunction		Page133
21.	CallFunctionF		Page134
22.	CallFunctionInt		Page135
23.	CallFunctionIntF		Page136
24.	CallFunctionReal		Page137
25.	CallFunctionRealF		Page138
	CArm		
1.	GetName		Page140
2.	GetALID		Page141
3.	GetLevel		Page142
4.	GetMessage		Page143
5.	GetDetailInfo		Page144
6.	GetDescription		Page145
7.	GetModule		Page146
8.	GetRecoverDescription		Page147
9.	GetPossibleRecovery		Page148
10.	IsPossibleRecover		Page149
11.	GetInfoAll		Page150



12.	Status		Page151
13.	SetEnable		Page152
14.	Result		Page153
15.	Post		Page154
16.	PostAny		Page155
17.	PostTh		Page156
18.	PostAnyTh		Page157
19.	PostF		Page158
20.	PostAnyF		Page159
21.	PostThF		Page160
22.	PostAnyThF		Page161
23.	Recover		Page162
24.	Clear		Page163



CSeqObject

1. BEGIN_OBJECT_ENUMERATION / END_OBJECT_ENUMERATION

● Description

- Sequence 내에서 사용될 Object를 enum으로 정의하는 구간이다.

● Syntax

BEGIN_OBJECT_ENUMERATION

DIO(dioName)

AIO(aioName)

SIO(sioName)

SEQ(seqName)

ALM(alarmName)

CDIO dioEnum("dioName");

CAIO aioEnum("aioName");

CSIO sioEnum("sioName");

CSequence seqEnum("seqName");

CArm alarmEnum("alarmName");

END_OBJECT_ENUMERATION

● Parameter

BEGIN_OBJECT_ENUMERATION : Object 정의 시작

END_OBJECT_ENUMERATION : Object 정의 끝

DIO(dioName) : Digital IO를 선언, dioName으로 object가 생성된다.

AIO(aioName) : Analog IO를 선언, aioName으로 object가 생성된다.

SIO(sioName) : String IO를 선언, sioName으로 object가 생성된다.

SEQ(seqName) : Sequence IO를 선언, seqName으로 object가 생성된다.

ALM(alarmName) : Alarm 을 선언, alarmName으로 object가 생성된다.

CDIO enum("name") : Digital IO를 선언 , enum으로 object가 생성된다.

CAIO enum("name") : Analog IO를 선언 , enum으로 object가 생성된다.

CSIO enum("name") : String IO를 선언 , enum으로 object가 생성된다.

CSequence enum("name") : Sequence를 선언 , enum으로 object가 생성된다.

CArm enum("name") : Alarm 를 선언 , enum 으로 object 가 생성된다.

● Return

- Nothing

● Sample

```
BEGIN_OBJECT_ENUMERATION
    CDIO SIMULATION_CONTROL("System.SimulMode");
    CAIO APC_PRESSURE("Apc.Pressure_Set");
    CSIO SYSTEM_TIME("System.hhmmss");
    CSequence PIN_CONTROL("OBJ_PIN_CONTROL");
    CAlarm PIN_UP_TMO("PinUpCtrlTimeout");

    DIO(Pin_Up_Ctrl)
    AIO(Pin_Up_Timeout)
    SIO(Process_Message)
    SEQ(OBJ_DOOR_CONTROL)
    ALM(PinDnCtrlTimeout)
END_OBJECT_ENUMERATION
```

2. ReplaceName

- **Description**

- CSeqObject object replace name

- **Syntax**

- void ReplaceName(const TCHAR* szTarget , const TCHAR* szNew);

- **Parameter**

- szTarget : replace target
 - szNew : replace new name

- **Return**

- Nothing

- **Sample**

- ReplaceName("%1", "PM1");

3. GetName

- **Description**

- CSeqObject object name read

- **Syntax**

- virtual const TCHAR* GetName() const;

- **Parameter**

- Nothing

- **Return**

- sequence object name

- **Sample**

- CSequence Seqenum("Sequence_name") :
printf("Sequence Name=[%s]\n", Seqenum.GetName());

4. ObjType

- **Description**

- CSeqObject object type read

- **Syntax**

- virtual int ObjType() const=0;

- **Parameter**

- Nothing

- **Return**

- Object Type:eCAIO/eCDIO/eCSIO/eCEnum/eCLookup/etc

- **Sample**

- CSequence Seqenum("Sequence_name") :

- If (Seqenum.ObjType== eCSeq) {

- printf("Object Name=[%s] is SequenceWn", Seqenum.GetName());

- }

CIOObject

1. GetName

- **Description**

- IO의 name read 함수.

- **Syntax**

- const TCHAR* GetName() const;

- **Parameter**

- Nothing

- **Return**

- IO name

- **Sample**

- CDIO dioenum("dio_name") :

- printf("IO Name=[%s]\n", dioenum.GetName());

2. GetFormat

- **Description**

- IO의 format read 함수.

- **Syntax**

- BOOL GetFormat(TCHAR* pszFormat) const;

- **Parameter**

- pszFormat : IO Format read(%d/%f)

- **Return**

- true:성공, false:실패

- **Sample**

- CDIO dioenum("dio_name") :
char Buffer[32];
dioenum.GetFormat(Buffer);

3. GetIDs

● Description

- IO의 ID값 read 함수

● Syntax

- `BOOL GetIDs( int &id1 , int &id2 , int &id3 , int &id4 ) const;`

● Parameter

- Id1 : IO 의 id1 값
- Id2 : IO 의 id2 값
- Id3 : IO 의 id3 값
- Id4 : IO 의 id4 값

● Return

- true:성공, false:실패

● Sample

```
CDIO dioenum( "dio_name" ) :  
int id1,id2,id3,id4;  
dioenum.GetIDs(id1, id2, id3, id4);
```

4. GetType

- **Description**

- IO의 type을 read 함수

- **Syntax**

- unsigned char GetType() const;

- **Parameter**

- Nothing

- **Return**

- IO Type

- **Sample**

```
CDIO dioenum( "dio_name") :  
Unsigned char type;  
Type=dioenum.GetType();  
if( Type==eCDIO) {  
    printf("IO Name=[%s] is DIO\n", dioenum.GetName());  
}
```

5. GetMode

- **Description**

- IO의 Mode를 나타내는 함수

- **Syntax**

- unsigned char GetMode() const;

- **Parameter**

- Nothing

- **Return**

- IO 의 Mode

- **Sample**

- CDIO dioenum("dio_name") :

- Unsigned char type;

- Type=dioenum.GetMode();

- if(Type==ePOLLING) {

- printf("IO Name=[%s] is polling mode\n", dioenum.GetName());

- }

6. GetElapsedTime

- **Description**

- Read된 IO의 경과 시간을 나타내는 함수

- **Syntax**

- DWORD GetElapsedTime() const;

- **Parameter**

- Nothing

- **Return**

- IO read 후 경과된 시간(msec)

- **Sample**

- CDIO dioenum("dio_name") :

- DWORD data;

- data=dioenum.GetElapsedTime ();

- printf("IO Name=[%s] is elapsed time=[%f]Wn", dioenum.GetName(), data);

7. GetIODriver

- **Description**

- IO object의 Driver get 함수

- **Syntax**

- CIODrv GetIODriver() const;

- **Parameter**

- Nothing

- **Return**

- CIODrv object

- **Sample**

- CDIO dioenum("dio_name") :

- CIODrv Adrv;

- Adrv = dioenum.GetIODriver();

8. Poll

- **Description**

- IO Polling

- **Syntax**

- `void Poll() const;`

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

- `CDIO dioenum( "dio_name") :`
`dioenum.Poll();`

9. PollIfElapsed

- **Description**

- 이전 read 시간이 지정시간 이상 지난 경우 IO Polling

- **Syntax**

```
void PollIfElapsed(int nPollTime) const;
```

- **Parameter**

- nPollTime: 경과시간(msec)

- **Return**

- Nothing

- **Sample**

```
CDIO dioenum( "dio_name" ) :  
dioenum.PollIfElapsed(500);
```

CEnum & CDIO

1. GetName

- **Description**

- enum object name read

- **Syntax**

- const TCHAR* GetName() const;

- **Parameter**

- Nothing

- **Return**

- enum object name

- **Sample**

- CDIO dioenum("dio_name") :
CEnum en;
en= dioenum.GetEnum();
printf("Enum Name=[%s]\n", en.GetName());

2. GetMaxValue

- **Description**

- enum object의 Max 값 read 함수

- **Syntax**

- int GetMaxValue() const;

- **Parameter**

- Nothing

- **Return**

- enum 의 Max 값

- **Sample**

- CDIO dioenum("dio_name") :

- CEnum en;

- en= dioenum.GetEnum();

- printf("Enum Name=[%s], Max=[%d]\n", en.GetName(), en.GetMaxValue());

3. GetEnumStr

- **Description**

- enum값의 문자열 read 함수

- **Syntax**

- const TCHAR* GetEnumStr(int data) const;

- **Parameter**

- data : enum 값

- **Return**

- enum 값의 문자열

- **Sample**

- CDIO dioenum("dio_name") :

- CEnum en;

- en= dioenum.GetEnum();

- printf("Enum Name=[%s], enum[1]=[%s]\n", en.GetName(), en.GetEnumStr(1));

4. GetEnumStrAll

● Description

- enum 값 전부를 문자열로 read 하는 함수

● Syntax

```
const TCHAR** GetEnumStrAll( int &nMaxEnum ) const;
```

● Parameter

- nMaxEnum : enum 값 최대치

● Return

- enum 값의 문자열

● Sample

```
CDIO dioenum( "dio_name" ) :
```

```
CEnum en;
```

```
en= dioenum.GetEnum();
```

```
printf("Enum Name=[%s], enum all=[%s]\n", en.GetName(), en.GetEnumStrAll(en.GetMaxValue()) );
```

5. GetEnumValue

● Description

- enum 문자열에 대한 enum값을 read 하는 함수

● Syntax

```
int GetEnumValue( const TCHAR* szEnumValue ) const;           // return -1 if not found
```

● Parameter

- szEnumValue : enum string

● Return

- enum 값

● Sample

```
CDIO dioenum( "dio_name" ) :  
CEnum en;  
en= dioenum.GetEnum();  
printf("Enum Name=[%s], enum OffOn=[%d]\n", en.GetName(), en.GetEnumValue("OffOn") );
```

6. GetEnum

- **Description**

- IO의 enum object get 함수

- **Syntax**

- CEnum GetEnum() const;

- **Parameter**

- Nothing

- **Return**

- Enum String

- **Sample**

- CDIO dioenum("dio_name") :

- CEnum en;

- en= dioenum.GetEnum();

7. GetMinMax

● Description

- Digital IO에 대한 Min/Max 값 read 함수.

● Syntax

- `BOOL GetMinMax( int &Min , int &Max ) const;`

● Parameter

- Min : IO에 대한 Min 값을 return 한다.
- Max: IO에 대한 Max 값을 return 한다.

● Return

- true :성공, false :실패

● Sample

```
CDIO dioenum( "dio_name" ) :  
int Min, Max;  
dioenum.GetMinMax(Min, Max);
```

8. SetMinMax

● Description

- Digital IO에 대한 Min/Max 값을 설정하는 함수.

● Syntax

- void SetMinMax(int Min , int Max); Parameter

● Parameter

- Min : Digital IO에 대한 Min 설정 값
- Max:.. Digital IO에 대한 Max 설정 값

● Return

- true :성공, false :실패

● Sample

```
CDIO dioenum( "dio_name" ) :  
int Min=0, Max=3;  
dioenum.SetMinMax(Min, Max);
```

9. Compare

- **Description**

- Digital IO에 대해 특정 값과 비교 하는 함수이다

- **Syntax**

- BOOL Compare(CIOObject::eOP op , int Comperand);

- **Parameter**

- eOP : 비교 연산자[enum { eLES/*<*/ , eLE/*<=*/ , eEQ/*==*/ , eNEQ/*!=*/ , eGE/*>=*/ , eGRT/*>*/ }]
- Comperand : 비교 값

- **Return**

- true : 비교 값 만족

- **Sample**

```
CDIO dioenum( "dio_name" ) :  
if(dioenum.Compare(eEQ, 1) ) {  
    printf("IO Value is 1...same ok\n");  
}
```

10. Less

- **Description**

- 입력한 수의 부등을 나타내는 함수 (<)

- **Syntax**

BOOL Less(int Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 작을 때
- false : 입력한 값보다 크거나 같을 때

- **Sample**

dioenum.Less(1);

11. LessEqual

- **Description**

- 입력한 수의 부등을 나타내는 함수 ($\leq$)

- **Syntax**

BOOL LessEqual(int Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 작거나 같을 때
- false : 입력한 값보다 클 때

- **Sample**

dioenum.LessEqual(1);

12. Equal

- **Description**

- 입력한 수의 부등을 나타내는 함수 (==)

- **Syntax**

BOOL Equal(int Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값과 같을 때
- false : 입력한 값보다 작거나 클 때

- **Sample**

dioenum.Equal (1);

13. NotEqual

- **Description**

- 입력한 수의 부등을 나타내는 함수 (!=)

- **Syntax**

BOOL NotEqual(int Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값과 다를 때
- false : 입력한 값과 같을 때

- **Sample**

dioenum.NotEqual (1);

14. GreaterEqual

- **Description**

- 입력한 수의 부등을 나타내는 함수 ($\geq$)

- **Syntax**

BOOL GreaterEqual(int Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 크거나 같을 때
- false : 입력한 값보다 작거나 다를 때

- **Sample**

dioenum.GreaterEqual (1);

15. Greater

● Description

- 입력한 수의 부등을 나타내는 함수 (>)

● Syntax

BOOL Greater(int Comperand);

// eGRT (>)

● Parameter

- Comperand : 비교할 숫자

● Return

- true : 입력한 값보다 작을 때
- false : 입력한 값보다 크거나 같을 때

● Sample

dioenum.Greater (1);

16. InRange

● Description

- 입력한 수의 부등을 나타내는 함수 ($\text{Begin} \leq \text{Data} \leq \text{End}$)

● Syntax

BOOL InRange(int Begin , int End);

● Parameter

- Begin : 비교할 숫자의 시작 값
- End : 비교할 숫자의 끝 값

● Return

- true : Data 값이 Begin 값과 같거나 End 값 사이에 있을 때
- false : Data 값이 Begin 값과 다르거나 End 값 사이에 있지 않을 때

● Sample

dioenum.InRange(1,10);

17. InRangeEx

● Description

- 입력한 수의 부등을 나타내는 함수 (Begin < Data < End)

● Syntax

- BOOL InRangeEx(int Begin , int End);

● Parameter

- Begin : 비교할 숫자의 시작 값
- End : 비교할 숫자의 끝 값

● Return

- true : Data 값이 Begin 값과 End 값 사이에 있을 때
- false : Data 값이 Begin 값과 End 값 사이에 있지 않을 때

● Sample

dioenum.InRangeEx(1,10);

18. InRangeExEnd

● Description

- 입력한 수의 부등을 나타내는 함수 ($\text{Begin} \leq \text{Data} < \text{End}$)

● Syntax

BOOL InRangeExEnd(int Begin , int End);

● Parameter

- Begin : 비교할 숫자의 시작 값
- End : 비교할 숫자의 끝 값

● Return

- true : Data 값이 Begin 값과 End 값 사이에 있을 때
- false : Data 값이 Begin 값과 End 값 사이에 있지 않을 때

● Sample

dioenum.InRangeExEnd(1,10);

19. InRangeExBegin

● Description

- 입력한 수의 부등을 나타내는 함수 ($\text{Begin} < \text{Data} \leq \text{End}$)

● Syntax

BOOL InRangeExBegin(int Begin , int End);

● Parameter

- Begin : 비교할 숫자의 시작 값
- End : 비교할 숫자의 끝 값

● Return

- true : Data 값이 Begin 값과 End 값 사이에 있을 때
- false : Data 값이 Begin 값과 End 값 사이에 있지 않을 때

● Sample

dioenum.InRangeExBegin(1,10);

20. Read

- **Description**

- Digital IO에 대한 값 read 함수

- **Syntax**

- `int Read( int &nResult ) const;`

- **Parameter**

- nResult : IO reading 실행 결과

- **Return**

- IO Reading 값

- **Sample**

- `int CommStatus;`
`dioenum.Read(CommStatus);`

21. ReadWithIDs

- **Description**

- Digital IO에 대한 값 read 함수(ID를 이용한 read 함수)

- **Syntax**

- ```
int ReadWithIDs(int id1 , int id2 , int id3 , int id4 , int &nResult) const;
```

- **Parameter**

- Id1/id2/id3/id4 : IO driver에 전달할 id 값

- **Return**

- IO Reading 값

- **Sample**

- ```
int Result;  
dioenum.ReadWithIDs(1,0,0,0, Result);
```

22. ReadIfElapsed

● Description

- Digital IO에 대한 값 read 함수(일정 시간이 지나지 않은 경우 memory 값 read)

● Syntax

```
int ReadIfElapsed( int milSec , int &nResult ) const;
```

● Parameter

- milSec : 경과시간 설정 값(msec)
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
int Result;  
dioenum.ReadIfElapsed (100, Result);
```

23. ReadAndGetStr

● Description

- Digital IO에 대한 값 및 IO format read 함수

● Syntax

```
int ReadAndGetStr( TCHAR* pszFomattedData , int &nResult ) const;
```

● Parameter

- pszFomattedData : Digital IO format read 변수(%d)
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
int Result;  
dioenum.ReadAndGetStr (szFormat, Result);
```

24. ReadAndGetStrIfElapsed

● Description

- Digital IO에 대한 값 및 IO format read 함수
설정 경과시간이 되기 전에 IO가 read 되면 memory 값을 리턴한다.

● Syntax

```
int ReadAndGetStrIfElapsed( TCHAR* pszFomattedData , int milSec , int &nResult ) const;
```

● Parameter

- pszFomattedData : Digital IO format read
- milSec : 경과시간 설정 값(msec)
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
int Result;  
dioenum.ReadAndGetStrIfElapsed (szFormat, 100, Result);
```

25. ReadEnum

● Description

- Digital IO의 enum string에 대한 정의 값 read 함수

● Syntax

```
int ReadEnum( TCHAR* pszEnum , int &nResult ) const;
```

● Parameter

- pszEnum : Digital IO의 값에 대한 enum string
- nResult : IO reading 실행 결과

● Return

- IO enum 에 대한 값

● Sample

```
int Result, iData;  
iData= dioenum.ReadEnum (_TEXT("eOn"), Result);
```

26. ReadEnumIfElapsed

● Description

- Digital IO에 대한 값과 enum string read 함수
설정 경과시간이 되기 전에 IO가 read 되면 memory 값을 리턴한다

● Syntax

- int ReadEnumIfElapsed(TCHAR* pszEnum , int milSec , int &nResult) const;

● Parameter

- pszEnum: Digital IO enum string read 변수
- milSec : 경과시간 설정 값(msec)
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
Int Result;  
dioenum.ReadEnum (pszEnum, 100 , Result);
```

27. ReadFromMem

- **Description**

- Digital IO에 대한 값을 memory로 부터 read 하는 함수
IO를 Memory 로 부터 read 한다

- **Syntax**

int ReadFromMem() const;

- **Parameter**

- Nothing

- **Return**

- IO Memory Reading 값

- **Sample**

dioenum.ReadFromMem();

28. ReadAndGetStrFromMem

- **Description**

- Digital IO에 대한 IO값 및 format을 memory에서 read 하는 함수

- **Syntax**

- int ReadAndGetStrFromMem(TCHAR* pszFomattedData) const;

- **Parameter**

- pszFomattedData : IO format read 변수

- **Return**

- IO Memory Reading 값

- **Sample**

- int iData;

- iData= dioenum.ReadAndGetStrFromMem (pszFomattedData);

29. ReadEnumFromMem

- **Description**

- Digital IO의 enum string 에 대한 값을 memory에서 read 함수

- **Syntax**

- int ReadEnumFromMem(TCHAR* pszEnum) const;

- **Parameter**

- pszEnum : Digital IO enum string

- **Return**

- IO Reading 값

- **Sample**

- int iData;

- iData= dioenum.ReadEnumFromMem (_TEXT("eON"));

30. Write

● Description

- Digital IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다린다
- 일반적으로 가장 많이 사용하는 setting 함수 이다

● Syntax

```
BOOL Write( int SetValue );  
void Write( int SetValue , int &nResult );
```

● Parameter

- SetValue : IO Set 값
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
dioenum.Write(1 , Result);
```

31. WriteWithIDs

● Description

- Digital IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다린다
- id를 이용하여 write 한다(IO cfg에 선언되지 않은 id도 가능 하다)

● Syntax

```
void WriteWithIDs( int id1 , int id2 , int id3 , int id4 , int SetValue , int &nResult );
```

● Parameter

- Id1/id2/id3/id4 : Driver에 전달 할 id 값
- SetValue : IO Set 값
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
dioenum.WriteWithIDs(1,0,0,0, 1 , Result);
```

32. WriteTh

● Description

- Digital IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

```
void WriteTh( int SetValue );
```

● Parameter

- SetValue : IO Set 값

● Return

- Nothing

● Sample

```
dioenum. WriteTh(1);
```

33. WriteThWithIDs

● Description

- Digital IO에 대한 값 Set 함수.(ID를 사용)
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

```
void WriteThWithIDs( int id1 , int id2 , int id3 , int id4 , int SetValue );
```

● Parameter

- Id1/id2/id3/id4 : Driver에 전달 할 id 값
- SetValue : IO Set 값

● Return

- Nothing

● Sample

```
dioenum. WriteThWithIDs(1,0,0,0,1);
```

34. WriteEnum

● Description

- Digital IO에 대한 값을 enum string으로 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

```
void WriteEnum( const TCHAR* szEnumStr , int &nResult );
```

● Parameter

- szEnumStr : IO Set 값(enum string)
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
dioenum. WriteEnum(_TEXT("eOn") , Result );
```

35. WriteToMem

● Description

- Digital IO에 대한 값을 memory에 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

```
void WriteToMem( int SetValue );
```

● Parameter

- SetValue : IO Set 값

● Return

- Nothing

● Sample

```
dioenum. WriteToMem (1);
```

36. WriteToMemDirect

● Description

- WriteToMem() 과 동일한 함수 이다.
- 단, io change event가 발생 하지 않는다

● Syntax

```
void WriteToMemDirect( int SetValue );
```

● Parameter

- SetValue : IO Set 값

● Return

- Nothing

● Sample

```
dioenum. WriteToMemDirect (1);
```

37. Increase

● Description

- Digital IO에 대한 현재 값에 설정값을 증가시켜 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

```
int Increase( int iInc , int &nResult );
```

● Parameter

- iInc: IO에 대한 증가치 설정 값
- nResult : IO setting 실행 결과

● Return

- IO setting 값

● Sample

```
int Result;
```

```
dioenum. Increase (3 , Result);
```

38. Decrease

● Description

- Digital IO에 대한 현재 값에 설정값을 감소시켜 Set 함수.
- 실행 결과가 return 되기를 기다린다.

● Syntax

```
int Decrease( int iDec , int &nResult );
```

● Parameter

- iDec: IO에 대한 감소치 설정 값
- nResult : IO setting 실행 결과

● Return

- IO setting 값

● Sample

```
int Result;  
dioenum.Decrease(3 , Result);
```

39. Multiply

● Description

- Digital IO에 대한 현재 값에 설정값을 곱셈 하여 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

int Multiply(double m , int &nResult);

● Parameter

m: IO에 대한 곱하기 설정 값
nResult : IO setting 실행 결과

● Return

- IO setting 값

● Sample

int Result;
dioenum. Multiply (3 , Result);

40. Divide

● Description

- Digital IO에 대한 현재 값에 설정값을 나눗셈 하여 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

```
int Divide( double d , int &nResult );
```

● Parameter

d: IO에 대한 나누기 설정 값
nResult : IO setting 실행 결과

● Return

- IO setting 값

● Sample

```
int Result;  
dioenum. Divide(3 , Result);
```

CLookUp & CAIO

1. GetName

- **Description**

- IO의 lookup name read 함수.

- **Syntax**

- const TCHAR* GetName() const;

- **Parameter**

- Nothing

- **Return**

- IO lookup name

- **Sample**

- CAIO aioenum("aio_name") :
CLookUp lu;
lu= aioenum.GetLookUp();
printf("Lookup Name=[%s]\n", lu.GetName());

2. GetDepth

- **Description**

- IO lookup depth read

- **Syntax**

- int GetDepth() const;

- **Parameter**

- Nothing

- **Return**

- lookup depth

- **Sample**

- CAIO aioenum("aio_name") :

- CLookUp lu;

- lu= aioenum.GetLookUp();

- printf("Lookup Name=[%s], depth=[%d]\n", lu.GetName(), lu.GetDepth());

3. GetConversionMode

- **Description**

- Analog IO에 대한 Conversion mode 을 read 함수

- **Syntax**

- `int GetConversionMode() const;`

- **Parameter**

- Noting

- **Return**

- 0:Notuse, 1:Direct, 2:Linear, 3:Lookup

- **Sample**

- CAIO aioenum("aio_name") :

- `int data;`

- `data=aioenum.GetConversionMode();`

4. GetLookUp

- **Description**

- Analog IO에 대한 Lookup object read 함수.

- **Syntax**

- CLookUp GetLookUp() const;

- **Parameter**

- Nothing

- **Return**

- CLookUp object

- **Sample**

- CAIO aioenum("aio_name") :
CLookUp lu;
lu=aioenum.GetLookUp();

5. GetUnit

- **Description**

- Analog IO에 대한 단위 read 함수.

- **Syntax**

- ```
BOOL GetUnit(TCHAR* pszUnit) const;
```

- **Parameter**

- pszUnit : IO 단위를 받아온다.

- **Return**

- true :성공, false :실패

- **Sample**

- ```
CAIO aioenum( "aio_name" ) :  
char Buffer[64];  
aioenum.GetUnit(Buffer);
```

6. GetMinMax

● Description

- Analog IO에 대한 Min/Max 값 read 함수.

● Syntax

- `BOOL GetMinMax(double& LMin , double& LMax , double& PMin , double& PMax ) const;`

● Parameter

- LMin : Analog IO에 대한 display Min 값을 return 한다.
- LMax: Analog IO에 대한 display Max 값을 return 한다.
- PMin : Analog IO에 대한 program Min 값을 return 한다.
- PMax : Analog IO에 대한 program Max 값을 return 한다.

● Return

- true :성공, false :실패

● Sample

CAIO aioenum("aio_name") :

Double lMix, lMax, pMin, pMax;

aioenum.GetMinMax(lMin, lMax, pMin, pMax);

7. SetMinMax

● Description

- Analog IO에 대한 Min/Max 값을 설정하는 함수

● Syntax

```
void SetMinMax( double LMin , double LMax , double PMin , double PMax );
```

● Parameter

- LMin : Analog IO에 대한 display Min 값을 return 한다.
- LMax: Analog IO에 대한 display Max 값을 return 한다.
- PMin : Analog IO에 대한 program Min 값을 return 한다.
- PMax : Analog IO에 대한 program Max 값을 return 한다.

● Return

- Nothing

● Sample

```
CAIO aioenum( "aio_name" ) :
```

```
Double lMix=0, lMax=100, pMin=0, pMax=1000;
```

```
aioenum.GetMinMax(lMin, lMax, pMin, pMax);
```

8. Compare

● Description

- Analog IO에 대해 비교값과 비교 연산자를 이용해 비교 하는 함수

● Syntax

```
BOOL Compare( CIOObject::eOP op , double Comperand );
```

● Parameter

- eOP : 비교 연산자
- Comperand : 비교값

● Return

- true :성공, false :실패

● Sample

```
if( aioenum.Compare(eLES, 100.0) ) {  
    printf("IO Name=[%s] is < 100.0Wn", aioenum.GetName());  
}
```

9. Less

- **Description**

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 (<)

- **Syntax**

- BOOL Less(double Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 작을 때
 - false : 입력한 값보다 크거나 같을 때

- **Sample**

- aioenum.Less(100.0);

10. LessEqual

- **Description**

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 ($\leq$)

- **Syntax**

- BOOL LessEqual(double Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 작거나 같을 때
 - false : 입력한 값보다 클 때

- **Sample**

- aioenum.LessEqual(100.0);

11. Equal

● Description

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 (==)

● Syntax

BOOL Equal(double Comperand);

// eEQ (==)

● Parameter

- Comperand : 비교할 숫자

● Return

- true : 입력한 값과 같을 때

- false : 입력한 값보다 작거나 클 때

● Sample

aioenum.Equal(100.0);

12. NotEqual

- **Description**

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 (!=)

- **Syntax**

- BOOL NotEqual(double Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값과 다를 때
 - false : 입력한 값과 같을 때

- **Sample**

- aioenum.NotEqual(100.0);

13. GreaterEqual

- **Description**

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 (>=)

- **Syntax**

- BOOL GreaterEqual(double Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 크거나 같을 때
 - false : 입력한 값보다 작거나 다를 때

- **Sample**

- aioenum.GreaterEqual (100.0);

14. Greater

- **Description**

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 (>)

- **Syntax**

- BOOL Greater(double Comperand);

- **Parameter**

- Comperand : 비교할 숫자

- **Return**

- true : 입력한 값보다 작을 때
 - false : 입력한 값보다 크거나 같을 때

- **Sample**

- aioenum.Greater(100.0);

15. InRange

- **Description**

- Analog IO에 대한 입력한 수의 부등을 나타내는 함수 ($\text{Begin} \leq \text{Data} \leq \text{End}$)

- **Syntax**

- `BOOL InRange( double Begin , double End );`

- **Parameter**

- Begin : 비교할 숫자의 시작값
 - End : 비교할 숫자의 끝값

- **Return**

- true : Data 값이 Begin 값과 같거나 End 값 사이에 있을 때
 - false : Data 값이 Begin 값과 다르거나 End 값 사이에 있지 않을 때

- **Sample**

- `aioenum.InRange(1.0, 100.0);`

16. InRangeEx

- **Description**

- Analog IO 에 대한 입력한 수의 부등을 나타내는 함수 (Begin < Data < End)

- **Syntax**

- BOOL InRangeEx(double Begin , double End);

- **Parameter**

- Begin : 비교할 숫자의 시작값
 - End : 비교할 숫자의 끝값

- **Return**

- true : Data 값이 Begin 값과 End 값 사이에 있을 때
 - false : Data 값이 Begin 값과 End 값 사이에 있지 않을 때

- **Sample**

- aioenum.InRangeEx(1.0, 100.0);

17. InRangeExEnd

- **Description**

- Analog IO 에 대한 입력한 수의 부등을 나타내는 함수 ($\text{Begin} \leq \text{Data} < \text{End}$)

- **Syntax**

- BOOL InRangeExEnd(double Begin , double End);

- **Parameter**

- Begin : 비교할 숫자의 시작값
 - End : 비교할 숫자의 끝값

- **Return**

- true : Data 값이 Begin 값과 End 값 사이에 있을 때
 - false : Data 값이 Begin 값과 End 값 사이에 있지 않을 때

- **Sample**

- aioenum.InRangeExEnd(1.0, 100.0);

18. InRangeExBegin

● Description

- Analog IO 에 대한 입력한 수의 부등을 나타내는 함수 ($\text{Begin} < \text{Data} \leq \text{End}$)

● Syntax

BOOL InRangeExBegin(double Begin , double End);

● Parameter

- Begin : 비교할 숫자의 시작값
- End : 비교할 숫자의 끝값

● Return

- true : Data 값이 Begin 값과 End 값 사이에 있을 때
- false : Data 값이 Begin 값과 End 값 사이에 있지 않을 때

● Sample

aiouenum.InRangeExBegin(1.0, 100.0);

19. Read

- **Description**

- Analog IO에 대한 값 read 함수

- **Syntax**

- double Read(int &nResult) const;

- **Parameter**

- nResult : IO reading 실행 결과

- **Return**

- IO Reading 값

- **Sample**

- int CommStatus;

- aioenum.Read(CommStatus);

20. ReadWithIDs

● Description

- Analog IO에 대한 값 read 함수(id를 이용해 read)

● Syntax

```
double ReadWithIDs( int id1 , int id2 , int id3 , int id4 , int &nResult ) const;
```

● Parameter

- Id1/id2/id3/id4 : IO driver에 전달 할 id 값

● Return

- IO Reading 값

● Sample

```
int Result;
```

```
aioenum. ReadWithIDs(1,0,0,0, Result);
```

21. ReadIfElapsed

● Description

- Analog IO에 대한 값 read 함수
- 일정 시간이 경과하지 않은 경우 memory 값 read

● Syntax

```
double ReadIfElapsed( int milSec , int &nResult ) const;
```

● Parameter

- milSec : 경과시간 설정 값(msec)
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
int Result;  
aioenum.ReadIfElapsed (100, Result);
```

22. ReadAndGetStr

● Description

- Analog IO에 대한 값 및 format read 함수

● Syntax

```
double ReadAndGetStr( TCHAR* pszFomattedData , int &nResult ) const;
```

● Parameter

- pszFomattedData : Analog IO format read 변수
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
int Result;
```

```
aioenum. ReadAndGetStr (TEST_1, Result);
```

23. ReadAndGetStrIfElapsed

● Description

- Analog IO에 대한 값 및 IO 이름 read 함수
- 설정 경과시간이 되기 전에 IO가 read 되면 Memory 값을 리턴한다

● Syntax

```
double ReadAndGetStrIfElapsed( TCHAR* pszFomattedData , int milSec , int &nResult ) const;
```

● Parameter

- pszFomattedData : Analog IO format read 변수
- milSec : 경과시간 설정 값(msec)
- nResult : IO reading 실행 결과

● Return

- IO Reading 값

● Sample

```
int Result;  
aioenum.ReadAndGetStr (TEST_1,100, Result);
```

24. ReadFromMem

- **Description**

- Analog IO에 대한 값을 Memory로 부터 read 함수
- IO를 Memory 로 부터 read 한다

- **Syntax**

double ReadFromMem() const;

- **Parameter**

- Nothing

- **Return**

- IO Reading 값

- **Sample**

aioenum. ReadFromMem();

25. ReadAndGetStrFromMem

- **Description**

- Analog IO에 대한 값 및 IO format을 Memory에서 read 함수

- **Syntax**

- double ReadAndGetStrFromMem(TCHAR* pszFomattedData) const;

- **Parameter**

- pszFomattedData : Analog IO format read 변수

- **Return**

- IO Reading 값

- **Sample**

- char Buffer[64;
aioenum. ReadAndGetStrFromMem(Buffer);

26. Write

● Description

- Analog IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다린다
- 일반적으로 가장 많이 사용하는 setting 함수 이다

● Syntax

```
BOOL Write( double SetValue );  
void Write( double SetValue , int &nResult );
```

● Parameter

- SetValue : IO Set 값
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
aioenum.Write(1.0 , Result);
```

27. WriteWithIDs

● Description

- Analog IO에 대한 값 Set 함수.(id를 이용하여 write)
- 실행 결과가 return 되기를 기다린다
- Driver에서 선언한 id를 이용하여 write 한다

● Syntax

```
void WriteWithIDs( int id1 , int id2 , int id3 , int id4 , double SetValue , int &nResult );
```

● Parameter

- Id1/id2/id3/id4 : IO driver에 전달 할 id 값
- SetValue : IO Set 값
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
aioenum.Write(1,0,0,0, 1 , Result);
```

28. WriteTh

● Description

- Analog IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

```
void WriteTh( double SetValue );
```

● Parameter

- SetValue : IO Set 값

● Return

- Nothing

● Sample

```
aioenum.WriteTh (1.0);
```

29. WriteThWithIDs

● Description

- Analog IO에 대한 값 Set 함수.(id를 이용하여 write)
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

- void WriteThWithIDs(int id1 , int id2 , int id3 , int id4 , double SetValue);

● Parameter

- Id1/id2/id3/id4 : Driver에서 정의한 id
- SetValue : IO Set 값

● Return

- Nothing

● Sample

aioenum.WriteThWithIDs (1,0,0,0,1);

30. WriteToMem

● Description

- Analog IO에 대한 값을 memory에 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

```
void WriteToMemDirect( double SetValue );
```

● Parameter

- SetValue : IO Set 값

● Return

- Nothing

● Sample

```
aioenum. WriteToMem (1);
```

31. Increase

● Description

- Analog IO에 대한 현재 값에 설정값을 증가시켜 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

```
double Increase( double dInc , int &nResult );
```

● Parameter

- iInc: IO에 대한 증가치 설정 값
- nResult : IO setting 실행 결과

● Return

- IO read 값

● Sample

```
int Result;
```

```
aioenum.Increase(1.0 , Result);
```

32. Decrease

● Description

- Analog IO에 대한 현재 값에 설정값을 감소시켜 Set 함수.
- 실행 결과가 return 되기를 기다린다.

● Syntax

```
double Decrease( double iDec , int &nResult );
```

● Parameter

- iDec: IO에 대한 감소치 설정 값
- nResult : IO setting 실행 결과

● Return

- IO read 값

● Sample

```
int Result;
```

```
aioenum.Decrease(1.0 , Result);
```

33. Multiply

● Description

- Analog IO에 대한 현재 값에 설정값을 곱셈 하여 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

```
double Multiply( double m , int &nResult );
```

● Parameter

- m: IO에 대한 곱하기 설정 값
- nResult : IO setting 실행 결과

● Return

- IO Setting 값

● Sample

```
int Result;
```

```
aioenum.Multiply(1.5 , Result);
```

34. Divide

● Description

- Analog IO에 대한 현재 값에 설정값을 나눗셈 하여 Set 함수.
- 실행 결과가 return 되기를 기다린다

● Syntax

```
double Divide( double d , int &nResult );
```

● Parameter

- d: IO에 대한 나누기 설정 값
- nResult : IO setting 실행 결과

● Return

- IO Setting 값

● Sample

```
int Result;  
aioenum.Divide (1.5 , Result);
```

CSIO

1. Compare

- **Description**

- String IO에 대한 비교값 과 비교 연산자를 이용하여 비교 하는 함수

- **Syntax**

- BOOL Compare(CIOObject::eOP op , const TCHAR* Comperand);

- **Parameter**

- eOP : 비교 연산자
 - Comperand : 비교값

- **Return**

- true:성공, false:실패

- **Sample**

- CSIO sioenum("sio_name") :
if(sioenum.Compare(eEQ, "ABCD") {
 printf("IO Name=[%s] value is ABCD\n", sioenum.GetName());
}

2. Equal

● Description

- String IO에 대한 입력한 문자열과 같은지 비교하는 함수

● Syntax

BOOL Equal(const TCHAR* Comperand);

● Parameter

- Comperand : 비교할 문자열

● Return

- true : 문자열이 같을 때
- false : 문자열이 다를 때

● Sample

```
CSIO sioenum( "sio_name" ) :  
if( sioenum.Equal( "ABCD" ) ) {  
    printf("IO Name=[%s] value is ABCD\n", sioenum.GetName());  
}
```

3. NotEqual

● Description

- String IO에 대한 입력한 문자열과 다른지 비교하는 함수

● Syntax

```
BOOL NotEqual( const TCHAR* Comperand );
```

● Parameter

- Comperand : 비교할 문자열

● Return

- true : 문자열이 다를 때
- false : 문자열이 같을 때

● Sample

```
CSIO sioenum( "sio_name" ) :  
if( sioenum.NotEqual( "ABCD" ) {  
    printf("IO Name=[%s] value is not ABCD\n", sioenum.GetName());  
}
```

4. Read

● Description

- String IO에 대한 값 read 함수
- 가장 일반적으로 사용되는 String Reading 함수 이다

● Syntax

```
TCHAR* Read( TCHAR* pszSIOData , int &nResult ) const;
```

● Parameter

- pszSIOData : IO reading 값
- nResult : IO reading 실행 결과

● Return

-

● Sample

```
char Buffer[256];  
int Result;  
sioenum.Read( Buffer , Result );
```

5. ReadWithIDs

● Description

- String IO에 대한 값 read 함수(ID를 이용하여 read)

● Syntax

TCHAR* ReadWithIDs(int id1 , int id2 , int id3 , int id4 , TCHAR* pszSIOData , int &nResult) const;

● Parameter

- Id1/id2/id3/id4 : IO driver에 전달 할 id 값
- pszSIOData : IO reading 값

● Return

- Nothing

● Sample

```
char Buffer[256];  
int Result;  
sioenum.ReadWithIDs( 1, 2, 3, 4, Buffer , Result );
```

6. ReadIfElapsed

● Description

- String IO에 대한 값 read 함수.
- 일정 시간이 경과하지 않은 경우 memory 값을 read

● Syntax

```
TCHAR* ReadIfElapsed( TCHAR* pszSIOData , int milSec , int &nResult ) const;
```

● Parameter

- pszSIOData : IO reading 값
- milSec : 경과시간 설정 값(msec)
- nResult : IO reading 실행 결과

● Return

-

● Sample

```
char Buffer[256];  
int Result;  
sioenum.ReadIfElapsed( Buffer , 500, Result );
```

7. ReadFromMem

- **Description**

- String IO에 대한 값을 Memory로 부터 read 함수

- **Syntax**

- TCHAR* ReadFromMem(TCHAR* pszSIOData) const;

- **Parameter**

- pszSIOData : IO reading 값

- **Return**

-

- **Sample**

- char Buffer[256];

- int Result;

- sioenum.ReadFromMem(Buffer);

8. Write

● Description

- String IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다린다.
- 일반적으로 가장 많이 사용하는 setting 함수 이다

● Syntax

```
BOOL Write( const TCHAR* szSetValue );  
void Write( const TCHAR* szSetValue , int &nResult );
```

● Parameter

- szSetValue : IO Set 값
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
sioenum.Write( "ABCD", Result );
```

9. WriteWithIDs

● Description

- String IO에 대한 값 Set 함수.(id를 이용하여 write)
- 실행 결과가 return 되기를 기다린다.
- id를 이용하여 write

● Syntax

```
BOOL WriteWithIDs( int id1 , int id2 , int id3 , int id4 , const TCHAR* szSetValue );  
void WriteWithIDs( int id1 , int id2 , int id3 , int id4 , const TCHAR* szSetValue , int &nResult );
```

● Parameter

- Id1/id2/id3/id4 : Driver에 전달 할 id 값
- szSetValue : IO Set 값
- nResult : IO setting 실행 결과

● Return

- Nothing

● Sample

```
int Result;  
sioenum.WriteWithIDs( 1, 2, 3, 4, "ABCD", Result );
```

10. WriteTh

● Description

- String IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다

● Syntax

```
void WriteTh( const TCHAR* szSetValue );
```

● Parameter

- szSetValue : IO Set 값

● Return

- Nothing

● Sample

```
int Result;  
sioenum.WriteTh( "ABCD" );
```

11. WriteThWithIDs

● Description

- String IO에 대한 값 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다.
- id를 이용하여 write 한다

● Syntax

```
void WriteThWithIDs( int id1 , int id2 , int id3 , int id4 , const TCHAR* szSetValue );
```

● Parameter

- Id1/id2/id3/id4 : Driver에 전달 할 id 값
- szSetValue : IO Set 값

● Return

- Nothing

● Sample

```
int Result;  
sioenum.WriteThWithIDs(1,2,3,4, "ABCD" );
```

12. WriteToMem

● Description

- String IO에 대한 값을 memory에 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다.

● Syntax

```
void WriteToMem( const TCHAR* szSetValue );
```

● Parameter

- szSetValue : IO Set 값

● Return

- Nothing

● Sample

```
int Result;  
sioenum.WriteToMem( "ABCD" );
```

13. WriteToMemDirect

● Description

- String IO에 대한 값을 memory에 Set 함수.
- 실행 결과가 return 되기를 기다리지 않는다.
- IO change event 가 발생 하지 않음

● Syntax

- void WriteToMemDirect(const TCHAR* szSetValue);

● Parameter

- szSetValue : IO Set 값

● Return

- Nothing

● Sample

```
int Result;  
sioenum.WriteToMemDirect( "ABCD" );
```

14. WriteCount

- **Description**

- 문자열중 size 수(byte) 만큼만 write

- **Syntax**

BOOL WriteCount(const void* pBuf , int nSize);

- **Parameter**

- pBuf: write 문자열
- nSize : write byte 수

- **Return**

- true:성공, false:실패

- **Sample**

int Result;

sioenum.WriteCount("ABCD", 2);

CSeqObject

1. GetName

- **Description**

- Sequence의 name read 함수.

- **Syntax**

- const TCHAR* GetName() const;

- **Parameter**

- Nothing

- **Return**

- Sequence name read 값

- **Sample**

- CSequence seqenum (_TEXT("seq_name"));

- Printf("Sequence Name=[%s]\n", seqenum.GetName());

2. Load

- **Description**

- Sequence load 함수.

- **Syntax**

- BOOL Load();

- **Parameter**

- Nothing

- **Return**

- true :성공, false :실패

- **Sample**

- CSequence seqenum (_TEXT("seq_name"));
seqenum.Load();

3. Kill

- **Description**

- Sequence Kill 함수.

- **Syntax**

- BOOL Kill();

- **Parameter**

- Nothing

- **Return**

- true :성공, false :실패

- **Sample**

- CSequence seqenum (_TEXT("seq_name"));
seqenum.Kill();

4. Status

- **Description**

- Sequence status read 함수.

- **Syntax**

- SEQ_STATUS Status() const;

- **Parameter**

- Nothing

- **Return**

- sequence status :SEQ_SUCCESS/SEQ_ABORT/SEQ_ERROR/SEQ_RUNNING/SEQ_FAIL)

- **Sample**

- CSequence seqenum (_TEXT("seq_name"));

- SEQ_STATUS seqsts;

- seqsts=seqenum.Status();

6. Write

● Description

- Sequence write 함수.

● Syntax

```
void Write( const TCHAR* szCmd , const void *pParam=0x00 , int nNumOfBytesToSend=0 );
```

● Parameter

- szCmd : Sequence 전달할 command string
- *Param: Sequence 전달할 constant 값의 pointer
- nNumOfBytesToSend ; pParam의 byte 수

● Return

- Nothing

● Sample

```
CSequence seqenum ( _TEXT("seq_name") );  
SEQ_STATUS seqsts;  
seqenum.Write("PICK", 0x00, 0);
```

7. Write2

- **Description**

- Sequence write 함수

- **Syntax**

- BOOL Write2(const TCHAR* szCmd , const void *pParam=0x00 , int nNumOfBytesToSend=0);

- **Parameter**

- szCmd : Sequence 전달할 command string
 - *Param: Sequence 전달할 constant 값의 pointer
 - nNumOfByteToSend ; pParam의 byte 수

- **Return**

- true :성공, false :실패

- **Sample**

- CSequence seqenum (_TEXT("seq_name"));

- BOOL seqsts;

- seqsts =seqenum.Write2("PICK", 0x00, 0);

8. WriteF

- **Description**

- Sequence write by format 함수

- **Syntax**

- BOOL WriteF(const TCHAR* szCmd , ...);

- **Parameter**

- **Return**

- true :성공, false :실패

- **Sample**

- ```
CSequence seqenum (_TEXT("seq_name"));
BOOL seqsts;
int i=10;
seqsts =seqenum.WriteF("PICK PM1 %d", i);
```

---

## 9. Run

---

### ● Description

- Sequence Run 함수.

### ● Syntax

```
SEQ_STATUS Run(const TCHAR* szCmd , const void *pParam=0x00 , int nNumOfBytesToSend=0);
```

### ● Parameter

- szCmd : Sequence 전달할 command string
- \*Param: Sequence 전달할 constant 값의 pointer
- nNumOfBytesToSend ; pParam의 byte 수

### ● Return

- typedef enum { SEQ\_UNKNOWN , SEQ\_SUCCESS , SEQ\_ABORT , SEQ\_ERROR , SEQ\_RUNNING , SEQ\_FAIL } SEQ\_STATUS;

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
seqsts=seqenum.Run("PICK", 0x00, 0);
```

---

## 10. RunF

---

- **Description**

- Sequence Run by format 함수.

- **Syntax**

- SEQ\_STATUS RunF( const TCHAR\* szCmd , ... );

- **Parameter**

- **Return**

- SEQ\_STATUS

- **Sample**

- CSequence seqenum ( \_TEXT("seq\_name") );  
SEQ\_STATUS seqsts;  
int i=10;  
seqsts=seqenum.RunF("PICK PM1 %d", i);

---

## 11. RunTh

---

### ● Description

- Sequence Thread Run 함수

### ● Syntax

```
void RunTh(const TCHAR* szCmd , const void *pParam=0x00 , int nNumOfBytesToSend=0);
```

### ● Parameter

- szCmd : Sequence 전달할 command string
- \*Param: Sequence 전달할 constant 값의 pointer
- nNumOfBytesToSend ; pParam의 byte 수

### ● Return

- Nothing

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
seqenum.RunTh("PICK", 0x00, 0);
```



---

## 12. RunThF

---

- **Description**

- Sequence Thread Run by format 함수

- **Syntax**

- void RunThF( const TCHAR\* szCmd , ... );

- **Parameter**

- **Return**

- Nothing

- **Sample**

- CSequence seqenum ( \_TEXT("seq\_name") );

- SEQ\_STATUS seqsts;

- int i=10;

- seqenum.RunThF("PICK PM1 %d", i);

---

## 13. RunAny

---

### ● Description

- Sequence Run Any 함수.
- Abort() 를 받아도 하위 Function을 Run 전달 됨

### ● Syntax

SEQ\_STATUS RunAny( const TCHAR\* szCmd , const void \*pParam=0x00 , int nNumOfBytesToSend=0 );

### ● Parameter

- szCmd : Sequence 전달할 command string
- \*Param: Sequence 전달할 constant 값의 pointer
- nNumOfByteToSend ; pParam의 byte 수

### ● Return

- SEQ\_STATUS

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
seqsts=seqenum.RunAny("PICK", 0x00, 0);
```

---

## 14. RunAnyF

---

### ● Description

- Sequence Run Any by format 함수.
- Abort() 를 받아도 하위 Function을 Run 진행

### ● Syntax

```
SEQ_STATUS RunAnyF(const TCHAR* szCmd , ...);
```

### ● Parameter

### ● Return

- SEQ\_STATUS

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
int i=10;
seqsts=seqenum.RunAnyF("PICK PM1 %d", i);
```

---

## 15. RunAnyTh

---

### ● Description

- Sequence Thread Run 함수.
- Abort() 를 받아도 하위 Function을 Thread Run 전달 됨

### ● Syntax

- void RunAnyTh( const TCHAR\* szCmd , const void \*pParam=0x00 , int nNumOfBytesToSend=0 );

### ● Parameter

- szCmd : Sequence 전달할 command string
- \*Param: Sequence 전달할 constant 값의 pointer
- nNumOfByteToSend ; pParam의 byte 수

### ● Return

- Nothing

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
seqenum.RunAnyTh("PICK", 0x00, 0);
```

---

## 16. RunAnyThF

---

### ● Description

- Sequence Thread Run by format 함수.
- Abort() 를 받아도 하위 Function을 Thread Run 진행

### ● Syntax

```
void RunAnyThF(const TCHAR* szCmd , ...);
```

### ● Parameter

### ● Return

- Nothing

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
int i=10;
seqenum.RunAnyThF("PICK PM1 %d", i);
```

---

## 17. Abort

---

- **Description**

- Sequence Abort 함수.

- **Syntax**

- void Abort();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

- seqenum.Abort()

---

## 18. SendEventMsg

---

### ● Description

- Sequence event 발생 함수

### ● Syntax

- void SendEventMsg( const TCHAR\* szEvt , const void \*pParam=0x00 , int nNumOfBytesToSend=0 ) const;

### ● Parameter

- szEvt : Sequence 전달할 event string
- \*Param: Sequence 전달할 constant 값의 pointer
- nNumOfBytesToSend ; pParam의 byte 수

### ● Return

- Nothing

### ● Sample

```
CSequence seqenum (_TEXT("seq_name"));
SEQ_STATUS seqsts;
seqenum.SendEventMsg("PICK", 0x00, 0);
```

---

## 19. SendEventMsgF

---

- **Description**

- Sequence event by format 발생 함수

- **Syntax**

- void SendEventMsgF( const TCHAR\* szEvt , ... ) const;

- **Parameter**

- **Return**

- **Sample**

- CSequence seqenum ( \_TEXT("seq\_name") );  
SEQ\_STATUS seqsts;  
int i=10;  
seqenum.SendEventMsgF("PICK PM1 %d", i);



## 20. CallFunction

### ● Description

- Sequence Call Function 함수
- Sequence 내의 임의의 값 reading 에 이용한다
- call 되는 sequence에는 아래와 같이 정의 되어 있어야 한다

```

 BOOL FunctionMain(TCHAR *szReturn , int nSize , const TCHAR* szFnName , const TCHAR* szParam ,
 void *pParam , int nNumOfBytesRecved) {
 Return TRUE;
 }
 //
 void OnLoadComplete(const TCHAR* szArgument){
 SetFunctionHandler(FunctionMain);
 }

```

### ● Syntax

```

 BOOL CallFunction(TCHAR* szReturn , const TCHAR* szFnName , const TCHAR* szParam , void *pParam , int
 nNumOfBytesToSend) const;
 BOOL CallFunction(TCHAR* szReturn , int nSize , const TCHAR* szFnName , const TCHAR* szParam , void
 *pParam , int nNumOfBytesToSend) const;

```

### ● Parameter

- szReturn :
- szFnName :
- szParam :
- pParam :
- nNumOfByteToSend :

### ● Return

- true:성공, false:실패

### ● Sample

---

## 21. CallFunctionF

---

### ● Description

- Sequence Call Function by format 함수

### ● Syntax

BOOL CallFunctionF( TCHAR\* szReturn , const TCHAR\* szFnName , const TCHAR\* szParam , ... ) const;

BOOL CallFunctionF( TCHAR\* szReturn , int nSize , const TCHAR\* szFnName , const TCHAR\* szParam , ... )  
const;

### ● Parameter

- szReturn :
- szFnName :
- szParam :

### ● Return

- true :성공, false :실패

### ● Sample

---

## 22. CallFunctionInt

---

- **Description**

-

- **Syntax**

```
int CallFunctionInt(int &Result , const TCHAR* szFnName , const TCHAR* szParam , void *pParam , int
nNumOfBytesToSend) const;
```

- **Parameter**

- Result :
- szFnName :
- szParam :
- pParam :
- nNumOfByteToSend :

- **Return**

-

- **Sample**

---

## 23. CallFunctionIntF

---

- **Description**

-

- **Syntax**

int CallFunctionIntF( int &Result , const TCHAR\* szFnName , const TCHAR\* szParam , ... ) const ;

- **Parameter**

- Result :
- szFnName :
- szParam :

- **Return**

-

- **Sample**

---

## 24. CallFunctionReal

---

- **Description**

-

- **Syntax**

```
double CallFunctionReal(int &Result , const TCHAR* szFnName , const TCHAR* szParam , void *pParam , int
 nNumOfBytesToSend) const;
```

- **Parameter**

- Result :
- szFnName :
- szParam :
- pParam :
- nNumOfByteToSend :

- **Return**

-

- **Sample**

---

## 25. CallFunctionRealF

---

- **Description**

-

- **Syntax**

```
double CallFunctionRealF(int &Result , const TCHAR* szFnName , const TCHAR* szParam , ...) const ;
```

- **Parameter**

- Result :
- szFnName :
- szParam :

- **Return**

-

- **Sample**

# CAlarm

---

# 1. GetName

---

- **Description**

- Alarm name read 함수.

- **Syntax**

- const TCHAR\* GetName() const;

- **Parameter**

- Nothing

- **Return**

- Alarm name

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s]\n", Almenum.GetName() );



---

## 2. GetALID

---

- **Description**

- Alarm 번호 read 함수.

- **Syntax**

- int GetALID();

- **Parameter**

- Nothing

- **Return**

- Alarm number

- **Sample**

- CAlarm Almenu ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], ALID=[%d]\n", Almenu.GetName(), Almenu.GetALID());

---

## 3. GetLevel

---

- **Description**

- Alarm Level read 함수.

- **Syntax**

- int GetLevel() const;

- **Parameter**

- Nothing

- **Return**

- Alarm Level

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], Level=[%d]\n", Almenum.GetName(), Almenum.GetLevel());

---

## 4. GetMessage

---

- **Description**

- Alarm message read 함수.

- **Syntax**

- `const TCHAR* GetMessage();`

- **Parameter**

- Nothing

- **Return**

- Alarm message

- **Sample**

- `CAlarm Almenu ( _TEXT("AlmName" ) );`

- `Printf("Alarm Name=[%s], Message=[%s]\n", Almenu.GetName(), Almenu.GetMessage());`

---

## 5. GetDetailInfo

---

- **Description**

- Alarm detail info read 함수.

- **Syntax**

- const TCHAR\* GetDetailInfo();

- **Parameter**

- Nothing

- **Return**

- Alarm detail info

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], Detail Info=[%s]\n", Almenum.GetName(), Almenum.GetDetailInfo());

---

## 6. GetDescription

---

- **Description**

- Alarm description info read 함수.

- **Syntax**

- const TCHAR\* GetDescription();

- **Parameter**

- Nothing

- **Return**

- Alarm description info

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], Description=[%s]\n", Almenum.GetName(), Almenum.GetDescription());

---

## 7. GetModule

---

- **Description**

- Alarm Module read 함수.

- **Syntax**

- const TCHAR\* GetModule();

- **Parameter**

- Nothing

- **Return**

- Alarm Module info

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], Module=[%s]\n", Almenum.GetName(), Almenum.GetModule());

---

## 8. GetRecoverDescription

---

- **Description**

- Alarm recovery description info read 함수.

- **Syntax**

- const TCHAR\* GetRecoverDescription( ALM\_RESULT nRecover );

- **Parameter**

- nRecover : recovery type

- **Return**

- Alarm recovery description info

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], Recover=[%d]\n", Almenum.GetName(), Almenum.GetRecoverDescription());

---

## 9. GetPossibleRecovery

---

- **Description**

- Alarm recovery possible type read 함수.

- **Syntax**

- BYTE GetPossibleRecovery();

- **Parameter**

- Nothing

- **Return**

- Alarm recovery possible type

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );

- Printf("Alarm Name=[%s], Possible=[%d]\n", Almenum.GetName(), Almenum.GetPossibleRecovery());



---

## 10. IsPossibleRecover

---

- **Description**

- Alarm recovery type is possible read 함수.

- **Syntax**

- BOOL IsPossibleRecover( ALM\_RESULT nRecover );

- **Parameter**

- nRecover : Alarm recovery type

- **Return**

- true : recovery possible

- **Sample**

- CAlarm Almenum ( \_TEXT("AlmName" ) );  
ALM\_RESULT Result;  
Almenum.IsPossibleRecover(Result);

---

## 11. GetInfoAll

---

- **Description**

- Alarm 에 대한 모든 정보 read 함수.

- **Syntax**

- BOOL GetInfoAll();

- **Parameter**

- Nothing

- **Return**

- true :성공, false :실패

- **Sample**

- CArm Almenu ( \_TEXT("AlmName" ) );  
ALM\_RESULT Result;  
Almenu.GetInfoAll();

---

## 12. Status

---

- **Description**

- Alarm 상태 read 함수.

- **Syntax**

- ALM\_STATUS Status() const;

- **Parameter**

- Nothing

- **Return**

- ALM\_STATUS(ALM\_UNKNOWN/ALM\_ENABLE/ALM\_DISABLE)

- **Sample**

- CAlarm Almenu ( \_TEXT("AlmName" ) );

- ALM\_RESULT Result;

- Result=Almenu.Status();

---

## 13. SetEnable

---

- **Description**

- Alarm 상태를 Enable로 설정 함수.

- **Syntax**

- `void SetEnable( BOOL fEnable );`

- **Parameter**

- fEnable : true is enable

- **Return**

- Disable되어 있는 Alarm을 Enable 한다

- **Sample**

- `CAlarm Almenu ( _TEXT("AlmName" ) );`

- `ALM_RESULT Result;`

- `Almenu.SetEnable(TRUE);`

---

## 14. Result

---

- **Description**

- Alarm result read 함수.

- **Syntax**

- ALM\_RESULT Result() const;

- **Parameter**

- Nothing

- **Return**

- ALM\_RESULT(ALM\_CLEAR/ALM\_IGNORE=2/ALM\_RETRY=4/ALM\_ABORT=8/ALM\_ERROR=16/ALM\_FAIL=32/  
ALM\_POSTED=64)

- **Sample**

- CAlarm Almenu ( \_TEXT("AlmName" ) );  
ALM\_RESULT Result;  
Result=Almenu.Result();

---

## 15. Post

---

### ● Description

- Alarm 발생 함수.
- 이미 발생된 Alarm이 있으면, 다시 Alarm을 post 하지 않는다.
- Alarm post후 user의 recovery action을 기다린다

### ● Syntax

ALM\_RESULT Post();

ALM\_RESULT Post( const TCHAR\* szNewMsg );

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- ALM\_RESULT

### ● Sample

CAlarm Almenu ( \_TEXT("AlmName" ) );

ALM\_RESULT Result;

Result=Almenu.Post(); or Result=Almenu.Post("PLC Communication Error");

---

## 16. PostAny

---

### ● Description

- Alarm 발생 함수.
- 이미 발생한 Alarm이 있어도, 다시 Alarm을 post 한다.
- Alarm post후 user의 recovery action을 기다린다

### ● Syntax

```
ALM_RESULT PostAny();
```

```
ALM_RESULT PostAny(const TCHAR* szNewMsg);
```

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- ALM\_RESULT

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
```

```
ALM_RESULT Result;
```

```
Result=Almenu.PostAny(); or Result=Almenu.PostAny("PLC Communication Error");
```

---

## 17. PostTh

---

### ● Description

- Alarm 발생 함수.
- 이미 발생된 Alarm이 있으면, 다시 Alarm을 post 하지 않는다.
- Alarm post후 user의 recovery action을 기다리지 않는다.

### ● Syntax

```
void PostTh();
void PostTh(const TCHAR* szNewMsg);
```

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- Nothing

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
ALM_RESULT Result;
Almenu.PostTh(); or Almenu.PostTh("PLC Communication Error");
```



---

## 18. PostAnyTh

---

### ● Description

- Alarm 발생 함수.
- 이미 발생된 Alarm이 있어도, 다시 Alarm을 post 한다.
- Alarm post후 user의 recovery action을 기다리지 않는다.

### ● Syntax

```
void PostAnyTh();
void PostAnyTh(const TCHAR* szNewMsg);
```

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- Nothing

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
ALM_RESULT Result;
Almenu.PostAnyTh(); or Almenu.PostAnyTh("PLC Communication Error");
```

---

## 19. PostF

---

### ● Description

- Alarm Message를 by format로 바꾸어 Alarm 발생 함수.
- 이미 발생된 Alarm이 있으면, 다시 Alarm을 post 하지 않는다.
- Alarm post후 user의 recovery action을 기다린다

### ● Syntax

ALM\_RESULT PostF( const TCHAR\* szNewMsg , ... );

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- ALM\_RESULT

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
ALM_RESULT Result;
int i=10;
Result=Almenu.PostF("PLC Communication Error Comport %d", i);
```

---

## 20. PostAnyF

---

### ● Description

- Alarm Message를 by format로 바꾸어 Alarm 발생 함수.
- 이미 발생된 Alarm이 있어도, 다시 Alarm을 post 한다.
- Alarm post후 user의 recovery action을 기다린다

### ● Syntax

ALM\_RESULT PostAnyF( const TCHAR\* szNewMsg , ... );

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- ALM\_RESULT

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
ALM_RESULT Result;
int i=10;
Result=Almenu.PostAnyF("PLC Communication Error Comport %d", i);
```

---

## 21. PostThF

---

### ● Description

- Alarm Message를 by format로 바꾸어 Alarm 발생 함수.
- 이미 발생된 Alarm이 있으면, 다시 Alarm을 post 하지 않는다.
- Alarm post후 user의 recovery action을 기다리지 않는다.

### ● Syntax

```
void PostThF(const TCHAR* szNewMsg , ...);
```

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- Nothing

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
ALM_RESULT Result;
int i=10;
Almenu.PostThF("PLC Communication Error Comport %d", i);
```

---

## 22. PostAnyThF

---

### ● Description

- Alarm Message를 by format로 바꾸어 Alarm 발생 함수.
- 이미 발생된 Alarm이 있어도, 다시 Alarm을 post 한다.
- Alarm post후 user의 recovery action을 기다리지 않는다.

### ● Syntax

```
void PostAnyThF(const TCHAR* szNewMsg , ...);
```

### ● Parameter

- szNewMsg : Alarm 내용

### ● Return

- Nothing

### ● Sample

```
CAlarm Almenu (_TEXT("AlmName"));
ALM_RESULT Result;
int i=10;
Almenu.PosAnyThF("PLC Communication Error Comport %d", i);
```

---

## 23. Recover

---

- **Description**

- Alarm recovery 함수.

- **Syntax**

- `void Recover( ALM_RESULT nRecover );`

- **Parameter**

- nRecover : Alarm recovery type

- **Return**

- Nothing

- **Sample**

- `CAlarm Almenu ( _TEXT("AlmName" ) );`  
`ALM_RESULT Result;`  
`int i=10;`  
`Almenu.Recover( ALM_ABORT );`

---

## 24. Clear

---

- **Description**

- Alarm clear 함수.

- **Syntax**

- void Clear();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

- CAalarm Almenu ( \_TEXT("AlmName" ) );  
ALM\_RESULT Result;  
int i=10;  
Almenu.Clear( );

---

URL : <http://www.gesolutions.co.kr/>

(주)게스

우 305-500

대전광역시 유성구 용산동 533-1번지 미건테크노월드Ⅱ C동 440호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

